Testing

This document describes the testing strategy, test infrastructure, and the programming concepts applied in the test suite.

Overview

The project uses pytest as its test framework. Tests live in the `tests/` directory and cover the database layer, export functionality, email notifications, the Flask dashboard (routes, authentication, roles, the web sign-in page), every kiosk screen and reusable widget, the virtual keyboard, the Tkinter application entry point, and the fake-data seeding script. The suite targets 100% statement coverage, enforced by `.coveragerc` (`fail_under = 100`).

Test configuration is in `pytest.ini`:

```
[pytest]
testpaths = tests
pythonpath = kiosk_signin tests
```

The `pythonpath` setting adds both `kiosk_signin` and `tests` to the Python path, allowing test files to import application modules directly (e.g. `import database`) and shared test helpers.

Running Tests

```
pytest
```

Or for verbose output:

```
pytest -v
```

Running Tests With Coverage

```
pytest --cov --cov-report=term-missing
```

`.coveragerc` (repo root) configures `coverage`/`pytest-cov`:

```
[run]
source = kiosk_signin, dashboard
omit = */tests/*, */__pycache__/*

[report]
show_missing = True
fail_under = 100
```

```
exclude_lines =
    pragma: no cover
    if __name__ == .__main__.:
```

`fail_under = 100` makes the coverage command itself fail (non-zero exit) if any line drops out of coverage -- this is what CI checks. The `exclude_lines` patterns exempt two categories of genuinely untestable code from the count: lines explicitly marked `# pragma: no cover` (used sparingly, for platform-specific branches like the PyInstaller-frozen `sys.frozen` paths in `database.py`/`dashboard/app.py`, which only execute inside a packaged executable and can't be exercised from a normal test run), and `if __name__ == "__main__":` entry-point guards.

Continuous Integration

`.github/workflows/test.yml` runs the full test suite with coverage on every push and pull request (separate from `.github/workflows/build.yml`, which only builds release binaries when a GitHub release is published -- see 12-build-and-deployment.md). Because a large share of the suite drives real Tkinter widgets, the CI job installs `python3-tk` and `xvfb` and runs pytest under `xvfb-run` so those tests execute for real in CI rather than silently auto-skipping (see the `tk_root` fixture below) -- keeping the CI coverage number honest and equal to what runs locally on a machine with a display.

Test Infrastructure

Database Fixture

The core test fixture is defined in `tests/conftest.py`:

```
@pytest.fixture
def db(tmp_path, monkeypatch):
    monkeypatch.setattr(db_module, "DB_PATH", str(tmp_path / "test.db"))
    db_module.init_db()
    return db_module
```

This fixture:

1. Uses pytest's `tmp_path` fixture to create a temporary directory that is automatically cleaned up after the test.
2. Uses `monkeypatch.setattr()` to redirect the `DB_PATH` constant in `database.py` to a temporary file. This means every test gets a fresh, isolated database.
3. Calls `init_db()` to set up the schema and seed data.
4. Returns the `database` module so tests can call its functions directly.

This approach is a form of dependency injection -- the production database path is replaced with a test-specific one at runtime, without modifying the source code.

Tk Root Fixture

Screen and widget tests that need a real Tk event loop (for hover/focus/click events to actually fire, not just be constructed) use the `tk_root` fixture, also in `tests/conftest.py`:

```
@pytest.fixture
def tk_root():
    try:
        import tkinter as tk
        root = tk.Tk()
        root.geometry("1x1+0+0")
        root.update()
        yield root
        root.destroy()
    except Exception:
        pytest.skip("no display available")
```

This creates a genuine (not withdrawn) 1x1 `Tk` window, positioned off in the corner so it's unobtrusive if a display is actually visible. Any exception during creation -- most commonly no X display / no `DISPLAY` environment variable on a headless machine -- causes the test to be skipped rather than failing the whole run, which is why CI installs Xvfb (see above) rather than relying on this skip path to keep the suite green.

Test Helpers

`tests/helpers.py` provides factory functions for creating test data:

```
def member(**overrides):
    defaults = {
        "first_name": "Jane",
        "last_name": "Smith",
        "date_of_birth": "2015-01-01",
        ...
    }
    return {**defaults, **overrides}
```

These use the dictionary merge pattern (`{**defaults, **overrides}`) to provide sensible defaults that can be selectively overridden per test. This keeps tests concise while ensuring all required fields are populated.

`helpers.py` also provides `FakeApp`, a stand-in for the kiosk's `App` object that every screen receives in its constructor. Screens only ever call `app.show_screen(name, **kwargs)` to navigate; `FakeApp` just records each call in a list instead of actually switching screens, so screen tests can assert on navigation (`fake_app.last_call == ("options", {})`) without needing a real multi-screen `App` wired up.

Test Modules

test_database.py

The largest test module, covering all database operations:

Member CRUD: - Insert, update, search, and delete members. - Verify cascading deletes (deleting a member removes their sign-ins and caregiver). - Test the `person_type` field for member/staff distinction. - Verify that search filters work correctly for each type.

PIN Verification: - Correct PIN returns True. - Incorrect PIN returns False. - Blank PIN allows any input (no restriction).

Sign-in/out: - Recording and retrieving sign events. - Getting the latest sign event (all time and today only). - Getting all of today's signs. - Export queries with date range filtering. - Verifying that member exports exclude staff and vice versa.

Direction Validation: - No signs today: allowed direction is IN. - After sign IN: allowed direction is OUT. - After sign OUT (any time of day): allowed direction is IN again -- no am/pm cutoff and no daily lockout. - Full day sequence test (IN, OUT, IN, OUT) still allows another IN afterwards. - Many IN/OUT cycles in a single day remain unrestricted. - Staff use the exact same unrestricted logic as members.

Admin Password: - Default password works. - Wrong password is rejected. - Password change updates the hash.

Run Groups: - Create a run sheet with members. - Re-saving the same name replaces the member list. - Search by name. - Delete a run sheet. - `get_run()` returns a run sheet by id, or `None` for an unknown id. - `list_runs()` includes each run's member count (including zero) and filters by a name query. - `create_run()`/`update_run()` (the id-keyed pair used by the dashboard's Run Sheets pages, as opposed to the kiosk's name-keyed `upsert_run()`): success cases, and `None`/`False` returned instead of raising when the name collides with a different run sheet; `update_run()` on an unknown id returns `False`. - `upsert_run()`, `create_run()`, `update_run()`, and `delete_run()` each write a `RUN_CREATED`/`RUN_UPDATED`/`RUN_DELETED` audit log entry; deleting an unknown id logs nothing. - `get_currently_signed_in()` returns each person's `id` alongside their name and sign-in time, excludes anyone whose latest event today is OUT, and returns an empty list when nobody is signed in.

Sign edit (admin time correction): - `get_sign()` returns the record joined with the member's name; unknown id returns `None`. - `update_sign_time()` changes the stored timestamp for that record only.

Dashboard Users (web dashboard login accounts): - `init_db()` seeds a default `admin`/`1234` account. - Creating a user hashes the password; a duplicate name returns `None` instead of raising; an optional `member_id` links the login to a staff record (or defaults to `None`). - `check_dashboard_login()` accepts the right name/password pair and rejects a wrong password or unknown name. - Listing returns users ordered by name; deleting removes the row; `set_dashboard_user_pin()` changes which password is accepted going forward. - `update_dashboard_user()` renames, re-roles, and re-links an existing login without touching its password; rejects a name collision with a different user (`False`) or an unknown id (`False`). - `validate_dashboard_password()` rejects passwords that are too short, missing a digit, or missing a special character, and accepts one that meets all three rules. - Account lockout: a failed login increments `failed_attempts`; the `LOCKOUT_THRESHOLD`th consecutive failure sets `locked_until`, which `is_dashboard_user_locked()` reports as locked until it's in the past; a successful login resets both fields to their unlocked defaults. - `check_admin_password()`/`set_admin_password()` (kiosk's separate `admins` table): a legacy unsalted-SHA-256 hash still authenticates, a wrong password against a legacy hash leaves it unchanged (not upgraded), and a correct password against a legacy hash transparently rewrites it in the new salted format -- then keeps authenticating normally afterward. - `get_signs_for_member()` now includes each event's `id` and `notes` in its result, needed to reuse `_pair_sign_in_visits()` for the self-service timesheet page.

test_dashboard.py

The largest test module by far, testing every Flask dashboard route via Flask's test client. Three fixtures build on each other:

```
@pytest.fixture
def client(db, monkeypatch):
   app.config["WTF_CSRF_ENABLED"] = False   # tested separately, see below
   limiter.enabled = False                  # tested separately, see below
   with app.test_client() as c:
       yield c

@pytest.fixture
def auth_client(client):
   # session["role"] = "admin"
   ...

@pytest.fixture
def staff_client(client, db):
   # creates a real dashboard_users row, session["role"] = "staff"
   ...
```

CSRF and rate limiting are disabled by default in `client` so the hundreds of other tests can POST plain form data the way they did before those protections existed -- they're re-enabled just for the handful of tests that specifically target them (see below).

Tests cover: - Login page rendering; successful and failed login attempts by name + password. - Dashboard page loads with analytics. - Logs page pagination and filtering. - Audit log page. - People listing and filtering by type. - Person creation, editing, and deletion. - Export downloads (verifying the response is a valid file in each supported format).

Multi-user login, roles, and route protection: - The seeded `admin`/`1234` login redirects to `/dashboard`; a staff login redirects to `/signin` instead. - Wrong password or unknown name shows a generic "Invalid name or password" error (no distinction between the two, to avoid confirming which names are valid accounts). - `session["role"]`, `session["name"]`, and `session["member_id"]` are set correctly after login. - `/` sends staff to `/signin` and admins to `/dashboard`. - Every admin-only route (`/dashboard`, `/logs`, `/audit`, `/people`, `/exports`, `/alerts`, `/dashboard-users`) rejects a `staff_client` with an "Admin access required" flash rather than a bare redirect loop or crash. - `/signin` (and the routes it needs) stay reachable for `staff_client`.

Editing a sign-in/out time: `_pair_sign_in_visits` now carries the underlying `sign_ins.id` for each side of a visit (`in_id`/`out_id`, `None` for whichever side is missing) so the Logs page can link to editing an individual record. Covered: - The paired-visit unit tests (`test_pair_visits_*`) assert `in_id`/`out_id` line up with the source rows. - The Logs page renders an "Edit" link for each side of a visit that has a recorded time. - `/logs/sign/<id>/edit` renders the form, accepts a valid `datetime-local` value and updates the record, rejects an invalid one without touching the stored timestamp, logs a `SIGN_TIME_EDITED` audit entry, 404s (via redirect + flash) on an unknown id, and requires `admin_required` (rejects `staff_client`).

Adding a missing sign-in/out time: the same `in_id`/`out_id` split makes it possible to tell exactly which side of a visit is missing, so the Logs page shows an "Add" link (instead of "Edit") for that side -- covering both a forgotten sign-out (an open visit) and a forgotten sign-in (a stray OUT with nothing preceding it).

Covered: - `test_logs_shows_add_link_for_open_visit` / `test_logs_shows_add_link_for_stray_out` -- the Logs page renders exactly one "Add" link (and one "Edit" link, for the recorded side) for each of the two missing-side cases; `test_logs_shows_edit_links_for_in_and_out` asserts a complete visit shows no "Add" links at all. - `test_sign_add_form_loads` / `test_sign_add_missing_member_id_redirects_with_flash` / `test_sign_add_invalid_direction_redirects_with_flash` / `test_sign_add_unknown_member_redirects_with_flash` -- `/logs/sign/add` validates its `member_id` and `direction` query params before rendering the form. The form-load test also checks the time defaults to just after the IN and the input's `min` is set. - `test_sign_add_requires_pair_record` / `test_sign_add_rejects_pair_from_another_person` -- the `pair_id` (the existing half of the visit) is required and must belong to the same person with the opposite direction. - `test_sign_add_creates_missing_out_for_open_visit` / `test_sign_add_creates_missing_in_for_stray_out` -- posting a valid `datetime-local` value inserts a new `sign_ins` row (via `database.add_sign()`) at that exact timestamp, for either direction. - `test_sign_add_rejects_invalid_timestamp` -- an invalid value is rejected without inserting anything. - `test_sign_add_logs_audit_event` -- a successful add logs a `SIGN_TIME_ADDED` audit entry. - `test_sign_add_requires_admin` -- the route rejects `staff_client`.

Keeping visits paired (regression for split visits): visits are rebuilt from time order, so an added/edited time on the wrong side of its partner used to split one visit into two rows. Covered: - `test_sign_add_rejects_out_before_its_in` / `test_sign_add_rejects_out_after_next_visit` / `test_sign_edit_rejects_moving_out_before_in` -- times outside the record's neighbours are rejected with a "stay part of this visit" message and nothing changes. - `test_sign_edit_rejects_moving_latest_record_before_previous` / `test_sign_edit_rejects_moving_first_record_after_next` -- the one-sided error messages when a record has only a previous (latest record) or only a next (first record) neighbour. - `test_get_sign_neighbours_unknown_id_returns_none_pair` / `test_delete_signs_empty_list_is_noop` (in `test_database.py`) -- the helpers' edge cases. - `test_sign_add_then_edit_keeps_single_visit` -- the reported scenario: add a missing OUT, edit it, and the Logs page still shows one row with two records in the database. - `test_logs_date_filter_keeps_visit_paired` -- the date filter is applied to whole visits (dated by their IN), so a visit crossing midnight isn't shown as a stray OUT. - `test_currently_signed_in_uses_time_order_not_insert_order` -- a back-dated IN added after a sign-out (higher `id`, earlier time) doesn't make the person show as signed in, for both `get_currently_signed_in()` and `get_allowed_direction()`.

Deleting sign-in/out records: `/logs/sign/delete` takes one or more `sign_id` values. Covered: - `test_logs_shows_delete_button_per_visit` / `test_sign_edit_page_has_delete_button` -- the buttons render. - `test_sign_delete_removes_whole_visit` / `test_sign_delete_single_record` -- both halves or a single record are removed, with a `SIGN_DELETED` audit entry. - `test_sign_delete_works_for_staff` -- staff records delete the same way as members. - `test_sign_delete_unknown_id_flashes` / `test_sign_delete_requires_admin` -- unknown ids flash "not found"; `staff_client` is rejected and nothing is deleted.

Run Sheets (web) and bulk sign-out: covers the dashboard's full run sheet management (`/run-sheets` and friends) plus the sign-in page's two bulk-select shortcuts, all reachable by `login_required` (not admin-only), matching the kiosk's lack of a role concept for run sheets: - `test_run_sheets_page_loads` / `test_run_sheets_page_shows_member_count` /

`test_run_sheets_search_filters_by_name` -- the list page and its search box. - `test_run_sheet_new_post_creates` / `test_run_sheet_new_post_missing_name_flashes_error` / `test_run_sheet_new_post_duplicate_name_flashes_error` -- creating a run sheet, including the friendly duplicate-name error instead of a raw database error. - `test_run_sheet_edit_get_prefills_members` / `test_run_sheet_edit_post_renames_and_updates_members` / `test_run_sheet_edit_post_duplicate_name_flashes_error` / `test_run_sheet_edit_unknown_id_flashes_not_found` -- editing, including renaming to a name another run sheet already has (rejected, original name preserved) and a 404-style "not found" flash for an unknown id. - `test_run_sheet_delete_removes_run` / `test_run_sheet_delete_unknown_id_flashes_not_found` -- deletion. - `test_run_sheet_search_endpoint_json` / `test_run_sheet_members_endpoint_json` / `test_run_sheet_members_endpoint_empty_run` -- the two JSON endpoints the sign-in page's "Load a Run Sheet" feature calls. - `test_run_sheets_available_to_staff_role` / `test_run_sheets_allow_staff` -- staff can reach and use every run sheet route. - `test_signin_currently_in_endpoint_json` / `test_signin_currently_in_excludes_signed_out` / `test_signin_currently_in_empty` -- the `/signin/currently-in` endpoint backing the "Select all currently signed in" button. - `test_add_sign_inserts_record_at_given_timestamp` (in `test_database.py`) -- direct unit test of `database.add_sign()`.

Logs search filter: the Logs page's `search` query param filters visits by member/staff name (case-insensitive substring match on first + last name), combinable with the existing status and date filters. Covered by `test_logs_search_filters_by_name`, `test_logs_search_matches_last_name`, `test_logs_search_is_case_insensitive`, `test_logs_search_no_match_shows_no_records`, and `test_logs_search_combines_with_status_filter`.

Staff "Who's Here" view: a `/status` route shares today's counts and currently-signed-in list with `/dashboard` via a small `_today_snapshot()` helper, but skips the historical charts and admin nav links. - Loads for both `staff_client` and `auth_client`; requires login (redirects an anonymous `client`). - Shows today's currently-signed-in people; has no People/Exports links. - The nav bar shows a "Who's Here" link for staff and not for admins (who use the full Dashboard link instead).

CSRF: - `test_csrf_protection_rejects_post_without_token` temporarily re-enables `WTF_CSRF_ENABLED` and confirms a POST without a `csrf_token` field is rejected (400), proving the protection is actually wired up and not just present in templates.

Rate limiting: - `test_login_rate_limited_after_repeated_failures` temporarily re-enables the limiter, exceeds 5 attempts/minute against `/login`, and confirms the 6th attempt gets HTTP 429.

Manage Dashboard Users: - Creating a user (with a role and an optional linked staff member) succeeds; a duplicate name is rejected with a friendly flash instead of a raw database error; a weak password is rejected with the specific rule that failed; an invalid role value silently falls back to `"staff"`. - Editing a user: the form loads pre-filled and 404-flashes on an unknown id; renaming/re-roling/re-linking works; leaving the password field blank keeps the current password, while a non-blank one changes it (and a weak one is rejected, leaving the old password in place); a duplicate name is rejected without changing the existing row; demoting the last remaining admin account to staff is blocked, mirroring the existing delete-the-last-admin protection. - Deleting a user works; deleting the sole remaining admin account is blocked (with a second admin present, deletion succeeds). - The whole page and every action (including edit) are admin-only.

Security hardening: - A failed login logs a `DASHBOARD_LOGIN_FAILED` audit entry (name attempted, never the password). - After `LOCKOUT_THRESHOLD` consecutive failures, further attempts -- even with the correct password -- are rejected with a distinct "temporarily locked" message, and no session is created. - Every response carries `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, and a `Content-Security-Policy` header, set by a global `@app.after_request` hook. - `Strict-Transport-Security` is present only when the request is HTTPS (simulated in tests via `base_url="https://localhost"`) and absent over plain HTTP -- both branches of that conditional are exercised.

My Timesheet (self-service): - An account with no linked staff member (`member_id` unset) gets a "not linked" flash and is redirected, for both the page and the download route. - A linked staff account sees their own paired sign-in/out sessions on the page, respects an optional `date_from`/`date_to` filter, and can download the same range in any of the three formats. - **Tamper-resistance**: `test_my_timesheet_never_reads_client_supplied_person_id` links one staff account to an empty-history person, gives a second person real sign-in records, then hits both `/my-timesheet` and `/my-timesheet/download` with a forged `person_id`/`member_id` query string pointing at the second person -- the response still reflects the *first* (empty) person's data, proving the routes never read those params at all rather than merely validating them. - The "My Timesheet" nav link appears only for a session with `member_id` set, and stays hidden for an unlinked admin session.

Web Sign In / Out page (`/signin`): - The page loads for both roles but requires login for anonymous visitors. - `/signin/search` finds members/staff by name and respects the `type` filter (a staff record never shows up in a member search and vice versa); an empty query returns no results. - `/signin/alerts` returns active alerts scoped to the given member ids, empty when there are none. - Submitting records the sign, respects `get_allowed_direction()` (a person already signed in is skipped rather than recorded again, with the rest of a batch still succeeding -- the same partial-success behaviour as the kiosk), rejects submissions with no one selected or an invalid direction, and fires the email notification exactly once per successfully-signed person (verified with `unittest.mock.Mock`, patched onto `email_notifier.send_notification`). - **Signature fidelity round-trip**: `test_signin_post_signature_round_trips_through_render_signature` POSTs synthetic JSON stroke data, reads back the stored `sign_ins.signature` blob, asserts it matches the exact byte format the kiosk's `SignatureCanvas.export_bytes()` produces, and then feeds it through `export_util.render_signature()` to confirm a real PNG comes back -- proving the browser-to-kiosk signature format bridge (see 06-flask-web-dashboard.md) actually works end-to-end, not just in isolation. - Malformed signature payloads (not JSON, not a list, wrong-length segments) are all rejected with a flash message and record nothing, rather than storing a blob that would silently fail to render later.

Staff vs. member `person_type` regression:

The `person_form.html` template is shared by both the "new member" and "new staff" screens, switched on a `person_type` variable. It once had no hidden `person_type` input, so a real browser submission always fell back to `app.py`'s `request.form.get("person_type", "member")` default -- every new "staff" record silently saved as a member, and the post-create redirect then rendered the member edit screen instead of the staff one. The fix adds:

```
<input type="hidden" name="person_type" value="{{ person_type }}">
```

The existing `test_create_staff` test didn't catch this because it hand-built the POST body with `"person_type": "staff"` already included, masking the missing field. To close that gap, `_rendered_form_fields()` (a small helper in `test_dashboard.py`) parses the actual `<input>` tags

out of the GET form's HTML and uses *that* as the POST body -- the way a real browser would, instead of a test-author's assumption of what the form contains. Covered by:

- `test_new_staff_form_includes_person_type_hidden_field` / `test_new_member_form_includes_person_type_hidden_field` -- the rendered form has the hidden field with the correct value.
- `test_create_staff_via_rendered_form_fields_only` / `test_create_member_via_rendered_form_fields_only` -- submitting exactly what the GET form renders creates a record with the correct `person_type`.
- `test_create_staff_redirects_to_staff_edit_view_not_member` -- after creating a staff member, the redirect target shows "Staff Details", not the member form ("Child Details" / "Caregiver").
- `test_created_staff_does_not_appear_in_member_list` / `test_created_member_does_not_appear_in_staff_list` -- the two lists stay mutually exclusive.

Exports page revamp (bulk member export removed, staff export gained a per-person mode):

The old "Member Signing Sheets" card and its `/export/members` route were removed -- `test_export_members_route_removed` checks the route now 404s, and `test_exports_page_loads` / `test_exports_page_has_no_bulk_member_route_reference` check the card and its form action are gone from the rendered page. The "Staff Signing Sheets" card was revamped into a single form that does either mode depending on whether a `person_id` is present:

- `test_export_staff_returns_xlsx` / `test_export_staff_missing_params` / `test_export_staff_empty_range` -- the original bulk "All Staff" behaviour (dates required) is unchanged.
- `test_exports_page_staff_dropdown_has_all_staff_option` -- the card's dropdown renders "All Staff" plus each staff member by name.
- `test_export_staff_specific_person_full_history` / `test_export_staff_specific_person_with_date_range` -- passing `person_id` exports just that staff member, with dates optional (blank means their entire history), reusing the same single-person logic as `/export/person`.
- `test_export_staff_specific_person_no_records_flashes_error` -- a staff member with no sign-ins yet flashes "No sign-in records found" instead of downloading an empty file.
- `test_export_staff_rejects_member_id` / `test_export_staff_rejects_unknown_person_id` -- passing a member's ID (or a nonexistent ID) to `/export/staff` is rejected with a "Staff member not found" flash rather than silently exporting the wrong person or crashing.

Export format options (xlsx/pdf/csv): `test_export_person_pdf_format` / `test_export_person_csv_format` / `test_export_staff_pdf_format` / `test_export_staff_csv_format` -- each single-person export route accepts `fmt=pdf`/`fmt=csv` and returns the right content type and file signature (`%PDF` magic bytes for PDF, `text/csv` content type for CSV). `test_export_staff_bulk_pdf_format` / `test_export_staff_bulk_csv_format` cover the same for `/export/staff`'s bulk (no `person_id`) branch, which dispatches on `fmt` separately from the shared single-person helper. `test_export_person_unknown_format_defaults_to_xlsx` / `test_export_staff_bulk_unknown_format_defaults_to_xlsx` -- an unrecognised `fmt` value falls back to Excel rather than erroring.

test_export.py

Tests the export generation for all three formats -- Excel, PDF, and CSV. `generate_export`/`generate_staff_export` (Excel), `generate_export_pdf`/`generate_staff_export_pdf`, and `generate_export_csv`/`generate_staff_export_csv` all share one underlying layout and the same `_build_staff_sessions()` grouping/pairing logic -- one section per person, one row per sign-in/out session -- so most of the tests below exercise that shared layout through whichever entry point is being tested.

- Member export and staff export each produce a valid .xlsx file, to a file path or to a `BytesIO` buffer.
- Empty data produces a file with a single placeholder sheet rather than an empty/broken workbook.
- Signature rendering produces PNG bytes from stroke data.
- Duration calculation handles various cases (normal, missing start/end, negative).

One worksheet per person (member/staff export unification): member and staff exports used to look different -- members got one sheet per calendar day with a fixed AM/PM grid, staff got a single sheet with a page-broken section per person. Both were unified onto the same one-sheet-per-person, one-row-per-session layout, driven by the shared `_generate_person_sessions_export()` / `_write_person_sheet()` functions. Covered: - `test_generate_export_one_sheet_per_member` / `test_generate_staff_export_one_sheet_per_staff_member` -- a multi-person export produces one worksheet named after each person, not one shared sheet. - `test_generate_export_multi_day_history_stays_on_one_sheet` -- a member's sign-ins spanning several days all land as separate session rows on their one sheet, rather than being split across per-day sheets (this is the modern equivalent of the old "multi-day ranges collapsed to a single row" regression test -- the failure mode moved, but the row-per-session assertion is the same idea). - `test_generate_export_embeds_signature_image` / `test_generate_staff_export_embeds_signature_image` -- a session with a captured signature gets a PNG embedded on the extra row underneath it.

CSV generation: `test_generate_export_csv_writes_rows` / `test_generate_staff_export_csv_writes_rows` -- a `StringIO` buffer ends up with a header row, one row per session, and a per-person "Total" row. `test_generate_export_csv_empty_signs_list` / `test_generate_export_csv_person_with_no_sessions` -- the no-data cases write a placeholder message instead of an empty or malformed file. `test_generate_export_csv_still_signed_in` -- an open session shows "Still signed in" in the Sign Out column, same wording as the Excel version.

PDF generation: each PDF test asserts the output starts with the `%PDF` magic bytes, since xhtml2pdf's `CreatePDF()` doesn't raise on most malformed-HTML input -- a missing/garbled magic number is the reliable failure signal. `test_generate_export_pdf_to_bytesio` / `test_generate_staff_export_pdf_to_bytesio` -- basic generation to a `BytesIO` buffer. `test_generate_export_pdf_multiple_people_paginates` -- more than one person's sessions still produce one valid PDF (the CSS `page-break-before` rule between people isn't independently checked, since it has no effect on whether the bytes are valid). `test_generate_export_pdf_empty_signs_list` / `test_generate_export_pdf_person_with_no_sessions` / `test_generate_export_pdf_still_signed_in` -- the same no-data and still-open-session cases as the Excel/CSV versions. `test_generate_export_pdf_escapes_html_special_characters` -- a name containing `<`/`&` still produces a valid PDF rather than broken markup, confirming `html.escape()` is actually applied to free-text fields before they're interpolated into the generated HTML. -

`_unique_sheet_title()` (`test_unique_sheet_title_*`) -- worksheet titles are stripped of Excel's reserved characters (`[ ] : * ? / \`), truncated to 31 characters, and de-duplicated (a second "Alice Brown" becomes "Alice Brown (2)") so two people who'd otherwise collide on a sheet name still each get their own tab.

Session pairing (`_build_staff_sessions`) -- nobody is restricted to one AM and one PM sign, so sessions are built by pairing IN/OUT events chronologically rather than by AM/PM slot (despite the `staff_` in its name, this function is shared by both the member and staff exports): - A single IN/OUT pair produces one session. - Multiple sessions in the same day are all kept, in order (not just the first). - Sessions are grouped correctly across multiple days. - People are sorted by last name. - A dangling IN with no matching OUT yet is kept with `out: None` ("still signed in"). - A stray OUT with nothing open is kept with `in: None` rather than silently dropped. - A second IN while one is already open closes the previous session with no sign-out, then opens a new one (defensive handling -- the kiosk itself never produces this sequence).

Person sheet layout (`_write_person_sheet`, shared by both exports): - A still-open session renders "Still signed in" in the sheet. - The "Total time in the centre" line at the end of a person's sheet sums every session's duration. - The name banner shows the year (or a year range, e.g. "2026–2027", when the exported data spans a calendar year boundary), plus the role for staff. - `_day_month()` formats dates as day/month with no year. - A person with no sessions in the exported range shows "No sign-in records for this period." instead of an empty row list.

test_email_notifier.py

Tests the email notification system using `unittest.mock` to avoid sending real emails:

```
@patch("email_notifier.smtplib.SMTP")
def test_send_notification(mock_smtp, db):
    ...
```

Tests cover: - Email is sent when a caregiver has an email address and notifications are enabled. - Email is not sent when notifications are disabled. - Email is not sent when the caregiver has no email. - Notification preference filtering (IN only, OUT only, BOTH, NONE).

test_profile.py

Tests member profile creation and update logic through the database layer: - Creating a member with all fields. - Updating specific fields. - Caregiver upsert (insert and update).

test_virtual_keyboard.py

Tests the virtual keyboard component without requiring a display: - Key press inserts characters. - Backspace deletes characters. - Shift toggles case. - Shift auto-releases after one character.

test_seed_fake_data.py

Tests `seed_fake_data.py`, the dummy-data generator used to populate a dev machine's database: - `_staff_day_sessions()` (the varied-time, multi-session staff day generator): every event falls on the requested day, events are chronologically ordered, session count stays within 1-3, at most the final session of a day is ever left open (a simulated forgotten sign-out), directions alternate correctly, and -- checked across many random seeds -- both multi-session days and open (still-signed-in) sessions do actually occur. - `_clear_previous_fake_data()` removes only "Fake"-named people, leaving real records untouched, and is a no-op (doesn't raise) when there's nothing to clear. - Running the seeding helpers twice back-to-back doesn't duplicate fake people (idempotency). - The full `_seed_history()` run actually produces staff days with more than one session -- a regression guard that the dummy data reflects the new unrestricted sign-in/out behaviour rather than the old one-shift-per-day pattern.

test_app.py

Tests the Tkinter entry point (`kiosk_signin/app.py`) -- the `App` class that wires up every screen and the `_start_dashboard()` function that launches the Flask dashboard. This is the one part of the kiosk that isn't a screen or a data-layer function, so it needs its own module:

- `App()` builds all ten expected screens (`tap`, `options`, `signin`, `signature`, `profile`, `social`, `run_sheets`, `admin_password`, `admin_panel`, `staff_profile`).
- On non-Windows platforms, construction skips the virtual keyboard (`self._vkb is None`) and doesn't try to maximise the window.
- On a simulated Windows platform (`_ON_WINDOWS` monkeypatched to `True`, with `VirtualKeyboard` stubbed out), construction creates a virtual keyboard bound to the same root window. The Windows-only `.state("zoomed")` call is stubbed in these tests too, since this environment's X server has no window manager to honour that wm state -- irrelevant to what's actually under test (that the Windows branch builds a keyboard).
- `show_screen()` lifts a known screen and calls its `on_show()`; an unknown screen name falls back to `"tap"` and prints a warning.
- Focus-driven virtual keyboard attach/detach: focusing an `Entry` widget attaches the keyboard; focusing a non-`Entry` widget does nothing; losing focus schedules a delayed detach, which a subsequent focus-in cancels; when nothing else grabs focus, the delayed detach actually runs and removes the keyboard.
- `run()` calls `self.root.mainloop()` -- verified by monkeypatching `mainloop` to a no-op that just records it was called, since actually entering the Tk event loop would hang the test.
- `_start_dashboard()` calls `waitress.serve()` with the real Flask dashboard app object and `host="0.0.0.0", port=5000` -- verified by monkeypatching `waitress.serve` to a recording stub, so the test never actually binds a port.

Every test that constructs a real `App()` uses the `db` fixture (so `App.__init__`'s `database.init_db()` call runs against an isolated temp-file database, never the developer's live one) and destroys `a.root` afterwards to avoid leaking Tk windows between tests.

Screen and Widget Tests

The remaining test modules exercise individual kiosk screens and reusable UI components, all built on the `tk_root` fixture (for screens/widgets that need a live event loop) and `FakeApp` (to assert on navigation without a real screen stack):

- **test_widgets.py** -- the shared component library (`kiosk_signin/components/widgets.py`): `TouchButton` hover/click behaviour and colour updates, `MutedLabel`/`FieldLabel`/`Divider` custom colours, `StyledEntry` get/set/clear, `DayPicker` toggle/multi-select state, plus `PopupOverlay` and `SignatureCanvas` (see below).
- **test_virtual_keyboard.py** -- the on-screen QWERTY keyboard: key press inserts characters, backspace deletes, shift toggles case and auto-releases after one keystroke.
- **test_splash_screen.py** -- `TapScreen` (tap-anywhere-to-start) and `OptionsScreen` (main menu): tapping navigates to the options menu, each menu button navigates to its correct screen, and `on_show()` never raises.
- **test_social_screen.py** -- the static Social Media & Contacts screen: constructs without error, back/exit buttons navigate to the options menu.
- **test_signin_screen.py** -- the member/staff search-and-select screen: name filtering (prefix and substring match, case-insensitive), adding/removing people from the multi-select list (duplicates ignored), and the search-results dropdown (already-selected people marked with a checkmark, clicking an unselected row adds them). `_add_member()` also clears the search text and refocuses the search entry -- the refocus is what keeps the on-screen keyboard attached (see app.py's `FocusIn`/`FocusOut` handlers) instead of it closing after every selection.
- **test_signature_screen.py** -- the IN/OUT + notes + signature capture screen: direction toggle colouring, populating the pupil list and auto-detecting the correct starting direction from `on_show()`, pre-filling notes from a single member's activity (but not when multiple members are selected), and `_submit()` recording signs only for members whose direction is currently allowed.
- **test_run_sheets_screen.py** -- bulk sign-in for a saved run group: run-name search/filtering, loading *all* of a run's members regardless of current sign status (a run sheet is used for signing a group out just as often as signing them in, so members currently signed IN must still load rather than being silently dropped, and no popup is shown for them), and the search dropdown's open/close/repopulate behaviour.
- **test_bus_sheets_screen.py** -- the read-only "who's signed in today" summary screen: empty-state message, listing today's sign-ins, refresh/reload behaviour, and back-button navigation.
- **test_profile_screen.py** -- the member profile edit screen: populating every field type from a database row (text, day pickers, boolean yes/no toggles, multi-line addresses, caregiver fields, notification preference), rendering the last-used signature (or a placeholder when none exists, or when the stored signature is garbage/unparseable), and the yes/no toggle widget's click behaviour.
- **test_staff_profile_screen.py** -- the staff profile edit screen: new-vs-existing staff title/delete-button state, populating fields, signature rendering (with the same placeholder/garbage-data fallbacks as the member profile screen), saving (create, update, and forcing `person_type="staff"`), required-field validation, and the delete-confirmation popup flow.
- **test_admin_screen.py** -- the largest screen test module: the admin password gate (correct/incorrect password, entry clearing on show), the admin panel's sign-in log view (empty state, listing, reload), password-change validation, and the export-range dialog (setting/clearing a date range, confirming with blank vs. valid vs. invalid dates, cancel-without-callback).

Programming Concepts Applied

Test Isolation

Each test runs with a completely fresh database. The `tmp_path` fixture creates a unique temporary directory per test, and `monkeypatch` redirects the database path. This ensures tests cannot interfere with each other regardless of execution order.

Fixtures and Dependency Injection

pytest fixtures provide test dependencies through function parameters. The `db` fixture encapsulates all database setup, and tests declare their dependency simply by including `db` as a parameter:

```
def test_upsert_member_insert(db):
    mid = db.upsert_member(_member())
    ...
```

This is dependency injection: the test receives its dependencies from the framework rather than creating them itself.

Factory Pattern (Test Helpers)

The `member()`, `caregiver()`, and `staff_member()` helper functions implement the factory pattern. They produce test data with sensible defaults, allowing tests to specify only the values relevant to what they are testing:

```
def test_search_by_name(db):
    mid = db.upsert_member(_member(first_name="Unique", last_name="Person"))
    results = db.search_members("Unique")
    assert any(m["id"] == mid for m in results)
```

Mocking (unittest.mock)

The email tests use `@patch` to replace the SMTP client with a mock object. This allows testing the notification logic without sending real emails or requiring network access:

```
@patch("email_notifier.smtplib.SMTP")
def test_send_notification(mock_smtp, db):
    mock_smtp.return_value.__enter__.return_value = MagicMock()
    ...
```

Arrange-Act-Assert

Tests follow the Arrange-Act-Assert pattern: 1. **Arrange**: Set up test data (create members, sign events, etc.). 2. **Act**: Call the function being tested. 3. **Assert**: Verify the result matches expectations.

Boundary Testing

The direction validation tests specifically target boundary conditions: - The start of day (no signs) -- allowed direction is IN. - Straight after a sign IN -- allowed direction is OUT. - Straight after a sign OUT, at any time of day -- allowed direction is IN again (no am/pm cutoff, no lockout). - A full day sequence from start to finish, and many IN/OUT cycles in a single day.